

LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

The Highest-Impact Optimization Layers for Local AI Inference

Report 01 · Core Optimization Survey | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

The Highest-Impact Optimization Layers for Local AI Inference

LocalHouseLLM Research

Executive Summary

Local AI inference now employs a vast array of hardware- and software-level optimizations to achieve low-latency, low-memory inference on edge devices. Key strategies include low-precision quantization, specialized attention kernels, graph compilers, batching and caching schemes, and system-level scheduling. For example, state-of-the-art quantization algorithms (GPTQ, AWQ, SmoothQuant, AQLM, HQQ, BitNet, etc.) compress LLM weights to 4, 2, or even ~ 1.58 bits per parameter, often with minimal accuracy loss. Hardware accelerators like NVIDIA's TensorRT-LLM apply custom kernels and precision tuning (e.g. INT4 AWQ, FP4/FP8) to exploit GPUs fully. System optimizations—such as prefix caching (reusing KV-caches for repeated prefixes), continuous batching (token-level scheduling for high throughput), chunked prefill (splitting long prompts to avoid decoding stalls), and speculative decoding (using a tiny “draft” model to parallelize token generation)—address latency and memory bottlenecks. Comprehensive engine frameworks (Torch Compile/Inductor, MLIR/TVM/IREE, ONNX Runtime, MLC-LLM, ExecuTorch, SGLang, etc.) integrate these techniques.

The following entries survey each technique in detail, with source citations. For each we list Name, Category, Description, Technical Basis, References, Repositories, Compatibility, Pros/Cons, and multidimensional ratings (stability, speed, memory, ease, etc). Because the field is fast-moving, ratings are qualitative (star scales and scores) with explanations. All claims are cross-checked against primary or authoritative sources when available. Uncertainties are noted. This compendium serves as an encyclopedia of local inference optimizations for AI systems engineers.

1. Quantization and Low-Precision Arithmetic

Quantization reduces model precision to cut memory and bandwidth. Techniques vary in method (post-training vs quantization-aware training), bit width (from FP16/BF16 down to 1–4 bits), and calibration needs. The table below highlights major algorithms and their trade-offs.

GPTQ (Post-Training Quantization) — Category: Weight-only PTQ (4-bit). First widely-used method to quantize large LMs to 4 bits while preserving accuracy. GPTQ uses Hessian information to minimally adjust weights into a 4-bit grid (no activation quant). Ref: Frantar et al. (2023). No speedup by itself (weights dequant on-the-fly), but 8x memory reduction vs FP16. Pros: Simple PTQ, high accuracy. Cons: No native low-bit matmul, so limited speed gain. Stability: ★★★★★ (established). Scores: Opt $\approx$ 8/10 (accuracy), Speed $\approx$ 1–2x (limited by dequant), Mem $\approx$ 8x reduction, Ease $\approx$ 7/10 (pipelines exist), Maturity high.

AWQ (Activations-aware W4A16 Quant) — Category: Post-Training (GPTQ variant). 4-bit weight quantization optimized for speed. AWQ (“AWQ: 1.41x Faster LLM inference with 4-bit quantization”) quantizes weights to 4-bit and keeps activations in FP16. In practice AWQ yields 3–4x speedups on GPU kernels vs FP16 (using fused 4-bit kernels) and $\sim$ 4x memory reduction, with near-zero accuracy loss. Pros: Faster inference, simple calibration. Cons: Hardware/engine support required for int4 kernels. Stability: ★★★★★ (widely integrated, HF supported). Opt Score: 9/10; Speed: $\sim$ 3x; Memory: $\sim$ 4x; Ease: 6/10; Maturity: medium-high.

SmoothQuant — Category: Mixed Precision PTQ. A method to enable 8-bit matrix multiplications by “smoothing” outliers into weights. SmoothQuant (Xiao et al. ICML 2023) applies a layer-wise linear transform that shifts large activations into weights, allowing weight+activation 8-bit quant (W8A8) with negligible loss. Achieves $\sim$ 1.56x throughput and $\sim$ 2x memory gain on FP16 models. Pros: No calibration data needed, easily applied. Cons: Only 8-bit, modest speedup (GPU compute-bound). Stability: ★★★★★; Opt: 7/10; Speed: $\sim$ 1.5x; Memory: $\sim$ 2x; Ease: 8/10; Maturity: medium.

Half-Quadratic Quantization (HQQ) — Category: Post-Training PTQ (1–8 bit, calibration-free). Proposed by Dropbox, HQQ minimizes quantization error via an iterative half-quadratic algorithm under a hyper-Laplacian prior, without any calibration data. Works for W8/W4 down to 1-bit, on any model (vision/NLP). Yields accuracy on-par with GPTQ/AWQ in 4-bit, and is faster: able to quantize a 70B LLM in $\sim$ 3 days (vs $\sim$ 10 days for AQLM). GitHub: dropbox/hqq. Pros: No calibration, multi-bit support, PyTorch 2.x+ compatible. Cons: New (2024), not as battle-tested. Stability: ★★☆☆☆; Opt: 8/10; Speed: $\sim$ 2x; Memory: up to 4x; Ease: 7/10; Maturity: emerging.

AQLM (Additive Quantization for LLMs) — Category: Post-Training PTQ. Multi-codebook quantization algorithm achieving extreme compression (down to 2 bits). AQLM splits weight matrices into groups and learns additive codebooks (1–3 codewords per group) to reconstruct weights. Sets new SOTA for 2-bit weight-only quant, beating GPTQ/QuIP in accuracy. Achieves 1.2–3x speedup when dequantization overhead is small. Repo: Vahe1994/AQLM on GitHub. Pros: Unmatched 2-bit accuracy, large memory saving (8x vs FP16). Cons: Very slow calibration (days on 70B), needs batch compute/GPUs. Stability: ★★☆☆☆ (research code); Opt: 9/10 (accuracy); Speed: $\approx$ 2x; Memory: 8x; Ease: 2/10; Maturity: low.

BitNet (1.58-bit Quant) — Category: Quantization-Aware Training (QAT). Architecture that forces each weight to be ternary $\{-1, 0, +1\}$, achieving a 1.58 bits/parameter representation. Introduced by Microsoft (Ma et al. 2024), BitNet b1.58 matches full-precision LLM performance while drastically reducing compute and energy (claimed $\sim$ 71x less arithmetic cost). Repo: MSR (bitnet/Triton kernels). Pros: Very high compression (x63 by size), massive energy reduction. Cons: Requires training from scratch or heavy fine-tuning; not compatible with off-the-shelf models. Stability: ★★☆☆☆ (very new); Opt: 10/10 (if training available); Speed: unknown; Memory: $\sim$ 63x; Ease: 1/10; Future: high potential.

Summary: These quantization methods exemplify a trade-off between complexity,

speed, and memory. Simpler PTQ (GPTQ, AWQ, HQQ) offers plug-and-play 4-bit models, whereas advanced methods (AQLM, BitNet) push to 2 or 1.58 bits at the cost of algorithmic complexity. Mixed-precision (e.g. FP16/BF16/FP8) is broadly supported in hardware and gives ~2x memory saving with minimal development effort. Tensor cores on modern GPUs accelerate FP16 and INT8, making low-bit inference practical.

2. Model Architecture Optimizations

Beyond numeric precision, altering the model structure itself can reduce inference cost. Techniques here include sparsity, pruning, mixture-of-experts (MoE), and dynamic skipping of computation.

Sparse Attention & Pruning (Token/Head Pruning, Early Exit): Some models incorporate mechanisms to skip attending to all tokens or layers. “Token pruning” methods drop low-importance tokens during decoding, and “early exit” allows an output if confidence is high. Sparse Attention (like BigBird or Reformer in research) limits attention range. These reduce computation but may complicate APIs. Stability: varies. Use Cases: specialized models (sparse-BERT, mixture-of-experts like GLaM/Qwen-MoE for routing sparsity).

Mixture-of-Experts (MoE): Models like Qwen-MoE, GPT-OSS, GLM-4.7 allocate multiple “expert” submodels and route tokens to a subset. Inference optimization includes expert prefetching and parameter-efficient structure. Example: “Speculating Experts” prefetches likely experts to overlap CPU-GPU memory I/O.

Model Distillation / Compression: Not local inference per se, but training smaller or distilled models (e.g. TinyBERT, DistILLM) is complementary.

3. Attention and Transformer Kernel Optimizations

Attention is often the bottleneck in transformers. Numerous kernel-level optimizations have been proposed:

FlashAttention (Wang et al. 2022): Recomputes attention in blocks to use GPU shared memory, drastically reducing memory bandwidth and enabling up to ~2-3x speedups on long sequences with lower memory. Torch and TensorRT now include FlashAttention kernels. Stability: ★★★★★☆; Opt: 8/10; Speed: 2-3x on long context; Memory: ~2x; Ease: 7/10; Community: high.

FlashAttention-2/3: Improved versions with better tiling and support for causal masking, enabling faster compute and lower memory (e.g. FlashAttention-2 claims 2-4x better speed on H100).

PagedAttention: Technique to stream attention blocks from memory in pages, reducing working set. Can interleave decode tokens with prefetch pages. Built into frameworks like TensorRT-LLM.

RadixAttention (SGLang) — SGLang’s backend caches common prefix tokens in a radix-tree to reuse KV caches across sessions. By reusing shared prefixes of prompts, it instantly detects when new prompts share a common start and avoids recomputation. Yields up to 6x higher throughput for templated prompts. Category: Caching optimization. Stability: ★★☆☆☆ (new DSL feature); Pros: Massive speedup for repeated prefixes; Cons: Added complexity.

4. Batching, Caching, and Scheduling Strategies

System-level scheduling can dramatically affect latency and throughput.

Static/Dynamic Batching (Baseline): Standard methods that wait for a batch of requests. Static batching gives high throughput but high latency. Dynamic batching trades some throughput for lower latency. Stability: ★★★★★.

Continuous Batching (a.k.a. In-flight Batching): Token-level batching that keeps GPUs fully utilized by replacing completed sequences with new ones immediately. VLLM and Tango (now SGLang's engine) implement continuous batching; NVIDIA's TensorRT-LLM uses "in-flight batching." Empirical results show ~8-23x throughput gains over naive dynamic batching. Stability: ★★★★★☆; Opt: 9/10; Speed: huge (10x+ in practice); Ease: 4/10; Adoption: growing.

Chunked Prefill: In continuous batching, a long prompt can stall decoding of others. Chunked prefill splits such prompts into smaller chunks processed over multiple iterations. Each chunk continues building the KV cache, avoiding blocking the pipeline. Stability: ★★★★★☆; Opt: 8/10; Use: emerging in vLLM and others.

Prefix Caching: Caches the KV-states of common prompt prefixes so that repeated queries can reuse prior work. Especially powerful for multi-turn dialogue or repeated system prompts. Stability: ★★★★★☆; Opt: 8/10; Benefit: Saves hours of decoding on repeated sessions.

KV-Cache Offloading: Moves cold/inactive KV states from GPU to CPU/RAM/disk. For long contexts or idle users, offloading frees GPU memory for other tasks without losing the cache. Stability: ★★★★★☆ (supported in Triton, vLLM); Opt: 7/10; Benefit: Significant memory relief when user sessions are intermittent.

Speculative Decoding: Runs a small "draft" model (e.g. 1-3B) in parallel with the full LLM to propose likely next tokens. The target model then validates or corrects them. Google's approach guarantees identical output and sees ~2-3x speedup. TensorRT-LLM also supports speculative decoding. Stability: ★★★★★☆; Opt: 8/10; Speed: ~2-3x; Quality: same by design; Ease: 5/10.

5. Compiler and Runtime Frameworks

Torch.compile (PyTorch Inductor) — PyTorch 2.x introduced torch.compile, which fuses kernels and enables CUDA graph capture. Inductor fuses chains of operations into single kernels, reducing launch overhead, and can capture the forward pass in a CUDA Graph. NVIDIA reports ~1.6x speedup in autoregressive decode at batch size 1 using CUDA graphs. Stability: ★★★★★☆; Opt: 8/10; Speed: 1.3-1.7x on low-concurrency workloads; Ease: 7/10.

XLA (Accelerated Linear Algebra) — Google's compiler originally for TF, now supports PyTorch/JAX. Fuses and optimizes kernels for target hardware (TPU/GPU). Less flexible for dynamic shapes; used mainly on TPU-based inference.

MLIR (Multi-Level IR) — A substrate for building domain-specific compilers (e.g. IREE). Provides infrastructure (dialects) to lower AI models to hardware code. Many projects (OpenXLA, TensorRT, IREE) build on MLIR.

TensorRT (NVIDIA) — Proprietary inference engine for NVIDIA GPUs. TensorRT-LLM is an open-sourced extension optimized for LLMs, incorporating flash attention, in-flight batching, paged KV cache, quantization, speculative decoding, etc. Supported on A100/H100, and now Jetson. Stability: ★★★★★ (production-ready); Opt: 10/10; Ease: 6/10; Adoption: high in GPU inference.

ONNX Runtime — Cross-platform engine (CPU/GPU/NNAPI) with graph optimizations (operator fusion, layer norm fusion, etc.). Good general performance, integrates with many hardware via plugins. Stability: ★★★★★; Opt: 7/10; Use: broad for vision/NLP models.

OpenVINO (Intel) — Inference toolkit for Intel CPUs/NPUs. Optimizes graph and uses MKL/DNNL kernels. Supports quantized models. Status: mainly vision, adding NLP support.

MLC-LLM — Open-source universal LLM engine that compiles models to efficient kernels on many platforms (CPU, CUDA, Apple M1/M2, Web). Has a unified runtime (MLCEngine) with an OpenAI-compatible API. Stability: ★★☆☆☆ (new); Opt: 7/10; Unique: wide platform coverage.

ExecuTorch (PyTorch mobile) — Meta’s on-device PyTorch solution. Exports PyTorch models to an intermediate .pte format via PyTorch’s export, then compiles/partitions them for various hardware. Powers billions of devices (WhatsApp, Quest, etc.). Stability: ★★★★★☆; Features: native PyTorch export, multi-backend, micro-runtime (~50KB); Use: mobile/IoT deployment.

SGLang — A new LLM workflow language and engine. A DSL for structured LLM programs, with primitives like gen(), fork(), join(). Its backend automatically optimizes scheduling (batching, caching). Features include RadixAttention (prefix caching) and FSM-based output validation. Stability: ★★☆☆☆ (early-stage); Adoption: limited but promising for complex pipelines.

Tinygrad — A minimal deep learning library. Not optimized for speed; mainly an educational/trivial framework, not a serious inference engine. Stability: ★☆☆☆☆.

6. Hardware-Specific and Edge Acceleration

Apple Metal Performance Shaders (MPS): macOS/iOS GPU API. PyTorch and MLC-LLM support MPS backend. Speeds up FP16/BF16 on Apple Silicon. Stability: ★★★★★☆ on recent Apple chips.

Vulkan Compute / DirectML / OpenCL: Low-level cross-platform compute APIs (Android, Windows, etc). Some engines (vLLM, PyTorch) can use Vulkan; DirectML enables GPUs on Windows. WebGPU (via WASM) is emerging for in-browser inference.

ROCm: AMD’s GPU platform. TensorRT-LLM recently added support for AMD MI250. TVM and PyTorch have ROCm backends.

FPGA/ASIC: Projects like Xilinx VitisAI, Intel OpenVINO for FPGAs/Movidius NPUs exist but are niche.

NPUs (Edge AI chips): Specialized chips (Qualcomm Hexagon, Apple Neural Engine, etc.) can run quantized networks via on-device ML frameworks. No general LLM compiler yet.

Memory Mapping & Weight Streaming: Some systems stream large model weights from disk/SSD (e.g. HuggingFace’s offload, DeepSpeed ZeRO offload) and compute partially to enable >GPU-memory models. Effect: trades I/O for capacity; complex to implement.

7. Key Findings and Gaps

Quantization is the dominant local optimization. Modern algorithms (GPTQ, AWQ, HQQ, AQLM, BitNet) have pushed from 8-bit down to ~ 1.6 -bit while preserving accuracy. However, there is an accuracy-speed gap: ultra-low-bit models often require full-precision dequantization or are slower if no dedicated kernels. Activation quantization (SmoothQuant, etc.) now enables weight+act 8-bit at scale. Mixed precision (FP16/BF16) remains a baseline.

Specialized kernels and compilers yield large gains: tensor fusion and CUDA Graphs give ~ 1.5 x speedups at low batch; FlashAttention gives ~ 2 – 3 x on long sequences; vendor runtimes (TensorRT-LLM) combine many tricks for 3x+ throughput boosts. However, these often require a specific stack or hardware. CPU inference is lagging: some frameworks (ONNX, MLC) help, but LLMs are still mainly GPU-bound unless heavily quantized.

Batching and scheduling are game-changers: naive single-request inference wastes GPU. Continuous batching in vLLM or Ray Serve can yield order-of-magnitude higher throughput. Prefix caching and speculative execution are also simple to implement in many serving systems and can multiply throughput for repeated or interactive use-cases.

Fragmentation and tooling gaps: Many solutions exist, but ecosystem maturity varies. Open-source tools are rapidly incorporating these (HuggingFace Transformers 4.38+ includes HQQ/AWQ/AQLM; PyTorch 2.6+ supports CUDA graphs; TensorRT-LLM is Open Source), but piecing them together is complex. Some advanced techniques (e.g. BitNet, RadixAttention) are very new and not production-ready. Unresolved: unified benchmarks comparing methods on real hardware, and user-friendly integration workflows. Also, “make it work on CPU” remains a challenge.

8. Practical Implications and Recommendations

Smallest Models (Edge, Mobile): Use QAT frameworks (ExecuTorch, TensorFlow Lite) with moderate quantization (8-bit) to fit on-device hardware. Leverage ExecuTorch for multi-architecture support.

GPU-based Local Servers: Aggressively apply 4-bit PTQ (AWQ/GPTQ) or mixed precision. Use high-speed inference stacks like TensorRT-LLM or vLLM with continuous batching. Enable prefix caching and KV-offloading in your service layer if workload warrants (multi-turn chat). Utilize torch.compile + CUDA graphs to reduce overhead.

Multi-user / Multi-tenant: Carefully manage GPU memory: offload idle KV caches, dynamically batch requests, and speculate experts (for MoE) or tokens (spec-decoding). Always measure both latency (TTFT) and throughput under your traffic pattern. Continuous batching and chunked prefill are particularly beneficial for non-fully-busy scenarios.

Emerging Tech: Keep an eye on sub-4-bit methods (AQLM, BitNet) for future on-device LLMs. Invest in frameworks that abstract precision (e.g. HuggingFace Accelerate with bitsandbytes, or MLC-LLM) to easily switch between kernels.

9. Future Work and Outlook

The trend is clear: larger models will demand even more extreme compression and smarter execution. Possible advances include dynamic/structured sparsity, hardware-designed 1-bit chips, and learning-based inference scheduling. The gap between research prototypes and robust, battle-tested libraries needs closing: e.g. AQLM and

RadixAttention are promising but immature. Continued benchmarking across hardware (GPUs, mobile SoCs, NPUs) is needed. Community efforts are documenting these fast, but standardized APIs (ONNX-LLM?) could unify the space.

Confidence: ★★★★★☆ overall. All techniques above are documented in publications or official docs; performance claims are cited where possible. Uncertainties remain for very new methods (e.g. BitNet’s real-world gains) and cross-framework compatibility issues.

References (prioritized)

1. Egiazarian et al., “Extreme Compression of LLMs via Additive Quantization (AQLM)” (ArXiv 2024).
2. Mekouri et al., “Fine-tuning LLMs to 1.58bit: extreme quantization” (HuggingFace blog, Sep 2024).
3. Yaniv Leviathan et al., “Looking back at speculative decoding” (Google AI Blog, Dec 2024).
4. NVIDIA Developer Docs, “TensorRT-LLM” (2024).
5. Half-Quadratic Quantization (Dropbox blog, 2024).
6. Anyscale Blog, “Continuous Batching: 23x throughput” (Jun 2023).
7. Modular Inference Handbook, “Prefix caching” (2024).
8. Modular Inference Handbook, “Static/Dynamic/Continuous Batching” (2024).
9. Mitrasish et al., “torch.compile and CUDA Graphs for LLM Inference” (Spheron, Apr 2026).
10. HF Transformers Docs, “BitNet” (2024).
11. Google Research Arxiv, “The Era of 1-bit LLMs: ... 1.58 Bits” (Ma et al. 2024).
12. HuggingFace Blog, “Why SGLang is a Game-Changer” (Aug 2024).
13. Madan et al., “Speculating Experts Accelerates MoE Inference” (ArXiv 2026).
14. HuggingFace Documentation, “MLC-LLM” (2024).
15. PyTorch ExecuTorch GitHub README.
16. HuggingFace Transformers Docs (Quantization section) — AWQ, HQQ, AQLM, BitNet (2023–24).
17. NVIDIA TensorRT-LLM GitHub (2024) — news and benchmarks.

Each rating was inferred from these and related sources. The list above is prioritized by relevance and authority (official docs, peer-reviewed papers, major blog posts).

LocalHouseLLM Research — Internal Technical Compendium